

## Patrones de cada Sprint y su ubicación en cada Sprint

### **Sprint1: Prefijo Base:**

PracticasEmpresariales/backend/src/main/java/co/edu/cole/practicas/

### **SINGLETON**

- config/singleton/SystemConfig.java
- audit/singleton/AuditoriaLogger.java
- config/SecurityConfig.java
- security/jwt/JwtUtil.java

### **PROXY**

- security/filter/ScopeValidationAspect.java
- security/jwt/JwtAuthenticationFilter.java
- security/annotation/RequiereRol.java
- security/annotation/SoloLectura.java

### **OBSERVER**

- event/AuditoriaEvent.java
- event/UsuarioCreadoEvent.java
- event/listener/AuditoriaEventListener.java
- event/listener/NotificacionEventListener.java

### **BUILDER**

- service/programa/ProgramaBuilder.java

### **MEDIATOR**

- service/dashboard/DashboardMediator.java
- service/dashboard/DashboardService.java
- controller/dashboard/DashboardController.java

## **Sprint 2:**

### **PROTOTYPE — GPE-141 / GPE-145**

- `src/main/java/co/edu/cue/practicas/pattern/prototype/IPrototype.java`
- `src/main/java/co/edu/cue/practicas/pattern/prototype/CatalogoPracticaPlantilla.java`
- `src/main/java/co/edu/cue/practicas/pattern/prototype/EmpresaPlantilla.java`

### **BUILDER — GPE-141 / GPE-143 / GPE-146 / GPE-150 / GPE-152**

- `src/main/java/co/edu/cue/practicas/pattern/builder/CatalogoPracticaBuilder.java`
- `src/main/java/co/edu/cue/practicas/pattern/builder/CatalogoPracticaDirector.java`
- `src/main/java/co/edu/cue/practicas/pattern/builder/EmpresaBuilder.java`
- `src/main/java/co/edu/cue/practicas/pattern/builder/VacanteBuilder.java`
- `src/main/java/co/edu/cue/practicas/pattern/builder/VacanteDirector.java`
- `src/main/java/co/edu/cue/practicas/pattern/builder/ExpedienteBuilder.java`

### **STRATEGY — GPE-145 / GPE-147**

- `src/main/java/co/edu/cue/practicas/pattern/strategy/EstrategiaValidacion.java`
- `src/main/java/co/edu/cue/practicas/pattern/strategy/EstrategiaValidacionEstandar.java`

### **CHAIN OF RESPONSIBILITY — GPE-145**

- `src/main/java/co/edu/cue/practicas/pattern/chain/ValidadorAptitud.java`

- src/main/java/co/edu/cue/practicas/pattern/chain/ContextoValidacion.java
- src/main/java/co/edu/cue/practicas/pattern/chain/ValidadorCatalogoActivo.java
- src/main/java/co/edu/cue/practicas/pattern/chain/ValidadorHojaDeVidaValida.java
- src/main/java/co/edu/cue/practicas/pattern/chain/ValidadorPracticaAnteriorFinalizada.java

### **TEMPLATE METHOD — GPE-145**

- src/main/java/co/edu/cue/practicas/pattern/template/PlantillaValidacionAptitud.java

### **PROXY — GPE-146 / GPE-147 / GPE-151**

- src/main/java/co/edu/cue/practicas/service/expediente/ExpedienteService.java (Protection + Cache Proxy)

### **OBSERVER — GPE-143 / GPE-147 / GPE-150 / GPE-151**

- src/main/java/co/edu/cue/practicas/event/EmpresaObserver.java
- src/main/java/co/edu/cue/practicas/event/TutorInactivacionObserver.java
- src/main/java/co/edu/cue/practicas/event/CoordinadorNotificacionObserver.java
- src/main/java/co/edu/cue/practicas/event/listener/ExpedienteEventListener.java

### **STATE — GPE-152 / GPE-153**

- src/main/java/co/edu/cue/practicas/model/entity/Vacante.java
- src/main/java/co/edu/cue/practicas/model/enums/EstadoVacante.java

## Sprint 3

### 1) OBSERVER (Patrón de Comportamiento)

Sí está implementado.

#### Archivos

- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/event/EmpresaObserver.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/event/CoordinadorNotificacionObserver.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/event/TutorInactivacionObserver.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/service/Empresa/EmpresaService.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/audit/singleton/AuditoriaLogger.java

#### Evidencia

• EmpresaObserver.java (línea 4): Define interfaz del observador con método onEmpresaEvento() que notifica sobre eventos: "EMPRESA\_APROBADA" | "EMPRESA\_RECHAZADA" | "EMPRESA\_INACTIVADA"

• CoordinadorNotificacionObserver.java (línea 7): Implementación concreta del Observer

que notifica al coordinador cuando ocurre un evento de empresa.

- TutorInactivacionObserver.java (línea 9): Implementación concreta que inactiva automáticamente

los tutores cuando la empresa se inactiva (regla GPE-151).

- EmpresaService.java (línea 61): Inyecta List<EmpresaObserver> y notifica a todos los observers cuando ocurren cambios de estado → SOLID DIP (Dependency Inversion Principle).

- AuditoriaLogger.java (línea 37): Utiliza ApplicationEventPublisher para publicar eventos de auditoría y que los listeners (observers) reaccionen (ej. alertas de seguridad).

## 2) SINGLETON (Patrón Creacional)

Sí está implementado.

### Archivos

- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/audit/singleton/AuditoriaLogger.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/config/singleton/SystemConfig.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/service/empresa/EmpresaService.java

### Evidencia

- AuditoriaLogger.java (línea 13): PATRÓN SINGLETON — GPE-139  
Instancia única que recibe eventos de todos los módulos del sistema.  
Centraliza el registro de acciones críticas garantizando trazabilidad completa.  
La bitácora es INMUTABLE: no se puede modificar ni eliminar entradas creadas.  
Regla de negocio: si falla el registro en bitácora, la operación se aborta.

- SystemConfig.java (línea 7): PATRÓN SINGLETON — GPE-136, GPE-140  
Configuración global del sistema leída de application.properties.  
Spring gestiona como bean singleton, garantizando única instancia en contexto de aplicación.  
Centraliza parámetros: app.nombre, app.universidad, app.jwt.expiration-ms, app.mail.\*

- EmpresaService.java (línea 42): PATRÓN SINGLETON

Spring gestiona este bean Service como singleton por defecto.

Anotación @Service + Bean management de Spring = Singleton implícito.

### 3) BUILDER (Patrón Creacional)

Sí está implementado.

#### Archivos

- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/pattern/builder/EmpresaBuilder.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/service/empresa/EmpresaService.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/model/entity/BitacoraAuditoria.java

#### Evidencia

- EmpresaBuilder.java (línea 11): PATRÓN BUILDER — ConcreteBuilder

Construye una Empresa paso a paso separando construcción de representación final.

Métodos: conRazonSocial(), conNit(), conSector(), conDireccion(), conTelefono(), conContacto(), conAreas(), build()

SOLID — SRP: solo construye Empresa. SOLID — OCP: nuevos campos = nuevos métodos.

- EmpresaService.java (línea 70): Uso de EmpresaBuilder en crearEmpresa()

Instancia builder y encadena métodos: new EmpresaBuilder()

.conRazonSocial("TechCo S.A.")

.conNit("900.123.456-7")

.conSector("Tecnología")

.conContacto("Juan", "juan@techco.com")

.build();

- BitacoraAuditoria.java: Builder patrón implícito via Lombok @Builder

Permite construcción fluida de entidades de auditoría.

## 4) PROTOTYPE (Patrón Creacional)

Sí está implementado.

### Archivos

- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/pattern/p  
rototype/EmpresaPlantilla.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/pattern/p  
rototype/IPrototype.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/service/e  
mpresa/EmpresaService.java

### Evidencia

- EmpresaPlantilla.java (línea 11): PATRÓN PROTOTYPE — ConcretePrototype  
Del diagrama: IPrototype → PracticeTemplate → (clona para crear) →

PracticeInstance

Cuando Coordinador registra empresa con estructura similar a existente, clona  
plantilla

y solo ajusta NIT y razón social. Clon siempre inicia en PENDIENTE.

SOLID — SRP: solo clona configuración. SOLID — OCP: nuevos campos  
clonables sin tocar.

- IPrototype.java: Interfaz genérica <T> define contrato clonar(): T

- EmpresaService.java (línea 95): Uso de EmpresaPlantilla en  
crearDesdeEmpresaExistente()

Empresa clon = new EmpresaPlantilla(origen).clonar();

Clona sector, dirección, municipio, teléfono, contacto, áreas — no NIT ni razón  
social.

## 5) PROXY (Patrón Estructural)

Sí está implementado.

### Archivos

- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/security/f  
ilter/ScopeValidationAspect.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/service/e  
xpediente/ExpedienteService.java

- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/model/entity/ExpedienteEstudiante.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/service/estudiante/EstudianteService.java

## Evidencia

- ScopeValidationAspect.java: Actúa como PROTECTION PROXY con @Aspect y @Around  
Intercepta todos los métodos anotados con @RequiereRol, bloqueando acceso por rol.  
Valida permiso ANTES de ejecutar: si falla, lanza 403 y registra en AuditoriaLogger.
- ExpedienteService.java: Usa CACHE/PROTECCIÓN PROXY para expedientes  
Aplica filtrado por rol en backend, también descrito como proxy en comentarios.
- ExpedienteEstudiante.java: Documentado como utilizando reglas de "inmutabilidad/proxy"  
Protege integridad de datos del expediente a través de validaciones.
- EstudianteService.java: Aplica filtrado por rol en backend, también descrito como proxy  
Implementa control de acceso por rol a operaciones de estudiante.

## 6) STATE (Patrón Comportamiento)

Sí está implementado.

## Archivos

- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/model/entity/InstanciaPractica.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/model/entity/PlanPractica.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/model/entity/SeguimientoSemanal.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/model/enums/EstadoPractica.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/model/enums/EstadoSeguimiento.java

## Evidencia

- InstanciaPractica.java: Implementa STATE para transiciones de práctica  
Estados: ASIGNADA\_PENDIENTE\_INICIO → EN\_CURSO → FINALIZADA/CANCELADA  
Métodos como confirmarVinculacion() validan estado actual antes de transicionar.  
Encapsula validaciones de transición dentro de la entidad.
- PlanPractica.java: Implementa STATE para plan de práctica  
Gestiona transiciones de estado con validaciones internas.
- SeguimientoSemanal.java: Implementa STATE para reporte de seguimiento  
Estados: PENDIENTE → APROBADO/RECHAZADO  
Métodos: aprobar(docenteld), rechazar(docenteld, motivoRechazo)  
Valida que estado sea PENDIENTE antes de transicionar, lanza OperacionNoPermitidaException si no.
- EstadoPractica.java (enum): Define estados permitidos para InstanciaPractica
- EstadoSeguimiento.java (enum): Define estados permitidos para SeguimientoSemanal

## 7) FACADE (Patrón Estructural)

Sí está implementado.

## Archivos

- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/service/vinculacion/VinculacionService.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/service/dashboard/DashboardIndicadorService.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/service/asignacion/AsignacionService.java

## Evidencia

- VinculacionService.java: FACADE que orquesta confirmación de vinculación de práctica  
Coordina múltiples operaciones: actualiza estado InstanciaPractica, persiste cambios,  
publica evento VinculacionConfirmadaEvent, registra auditoría.

Método public InstanciaPracticaResponse confirmarVinculacion() actúa como punto único de entrada simplificando para el cliente.

- DashboardIndicadorService.java: FACADE para agregación de métricas de dashboard

Calcula indicadores role-específicos desde múltiples repositorios en un solo método.

public DashboardIndicadores obtenerIndicadores() retorna datos consolidados sin exponer complejidad de múltiples consultas.

- AsignacionService.java: FACADE para orquestar proceso de asignación

Agrupar operaciones de creación, actualización y validación de asignaciones.

## **Sprint 4**

### **Template Method**

- service/evaluacion/AbstractEvaluacionFinalTemplate.java
- service/evaluacion/EvaluacionDocenteService.java
- service/evaluacion/EvaluacionTutorService.java

### **State**

- model/entity/EvaluacionFinal.java (método completar())
- model/entity/EncuestaSatisfaccion.java (métodos enviar(), guardarBorrador(), completar())
- model/entity/InstanciaPractica.java (método finalizarConResultado())

### **Observer**

- event/Sprint4DomainEvent.java
- event/listener/Sprint4ObserverListener.java

### **Chain of Responsibility + Decorator**

- service/cierre/ChecklistCierreService.java

### **Facade**

- service/cierre/CierreFormalFacade.java

## Proxy

- `model/entity/EvaluacionFinal.java` (bloqueo post-cierre en `completar()`)
- `model/entity/NotaFinalCoordinador.java` (bloqueo en `actualizar()`)

## Strategy

- `service/evaluacion/NotaFinalCoordinadorService.java`
- `service/configuracion/ProgramaConfiguracionService.java`

## Builder

- `service/reporte/ReporteEstadoProcesoService.java`
- `service/reporte/TableroGerencialDireccionService.java`

## Adapter

- `service/notificacion/NotificacionConfigurableService.java`
- `service/evaluacion/EvaluacionTutorService.java`

## Singleton

- `service/notificacion/NotificacionConfigurableService.java`

## Mediator

- `service/cierre/CierreFormalFacade.java` (coordinación de notificaciones)